Spark内存计算

丁玥
上海交通大学计算机学院
dingyue@sjtu.edu.cn

2026年秋季

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 Spark简介

2 Spark编程模型

3 SparkSQL

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



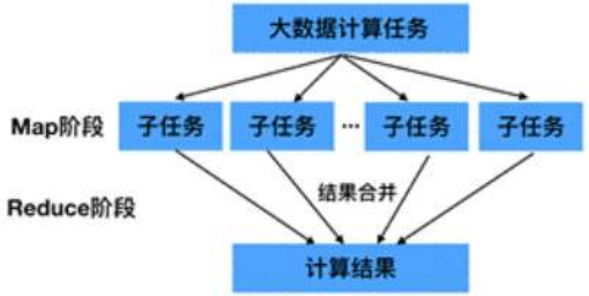
1 Spark简介

2 Spark编程模型

3 SparkSQL

Spark的诞生

- 在Hadoop出现之前，**分布式计算都是专用系统**，只能用来处理某一类的计算，比如进行大规模的排序。这样的系统无法复用到其他大数据计算场景。
- Hadoop MapReduce出现后，使得大数据计算通用编程成为可能，**只要遵循MapReduce编程模型编写业务处理代码，就可以运行在Hadoop分布式集群上**，而无需关心分布式计算是怎样完成的。



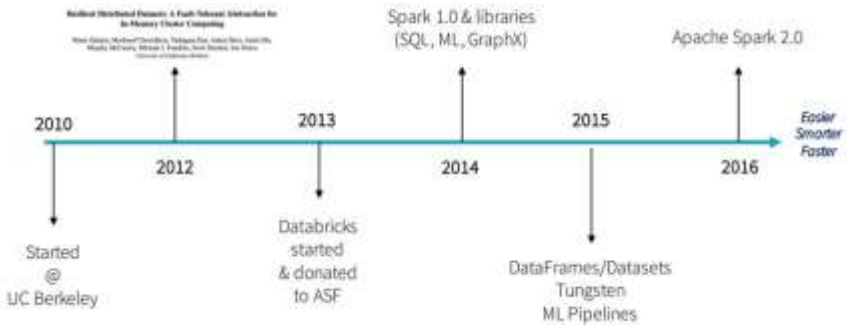
- MapReduce虽然已经可以满足大数据的应用场景，但是其**执行速度和编程复杂度并不让人们满意**。于是Spark应运而生

Spark简介

- Spark最初由美国加州伯克利大学的AMP实验室于2009年开发，是基于内存计算的大数据并行计算框架，可用于构建大型的、低延迟的数据分析应用程序

- 2013年Spark加入Apache孵化器项目后发展迅猛，如今已成为Apache软件基金会最重要的三大分布式计算系统开源项目之一（Hadoop、Spark、Storm）

- Spark在2014年打破了Hadoop保持的基准排序纪录
  - Spark/**206**个节点/**23**分钟/100TB数据
  - Hadoop/**2000**个节点/**72**分钟/100TB数据
  - Spark用1/10的计算资源，获得了比Hadoop快3倍的速度

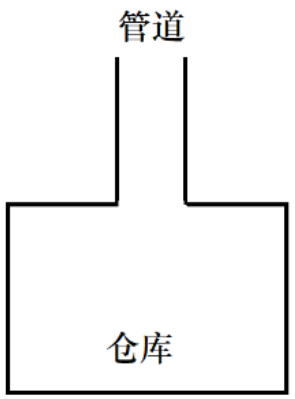


计算机的内存和磁盘

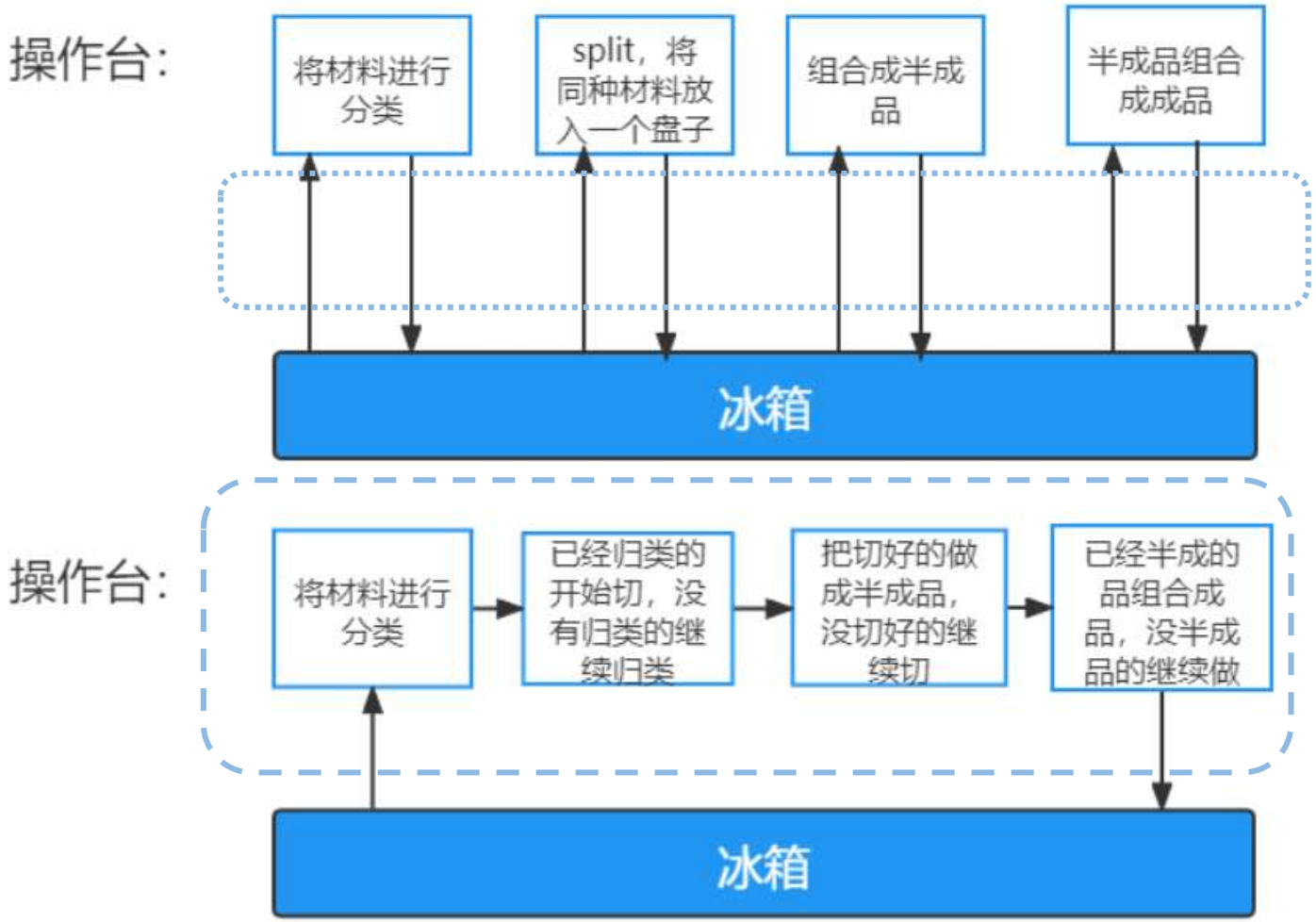
- **内存和磁盘两者都是存储设备**，但内存储存的是我们正在使用的资源，磁盘储存的是我们暂时用不到的资源。



- 可以把磁盘理解为一个仓库，而内存是进出这个仓库的通道。仓库（磁盘）很大，而通道（内存）很小，通道就很容易塞满。



结合我们之前讲的MapReduce的例子，假设把磁盘作为冰箱，内存为做饭时的操作台



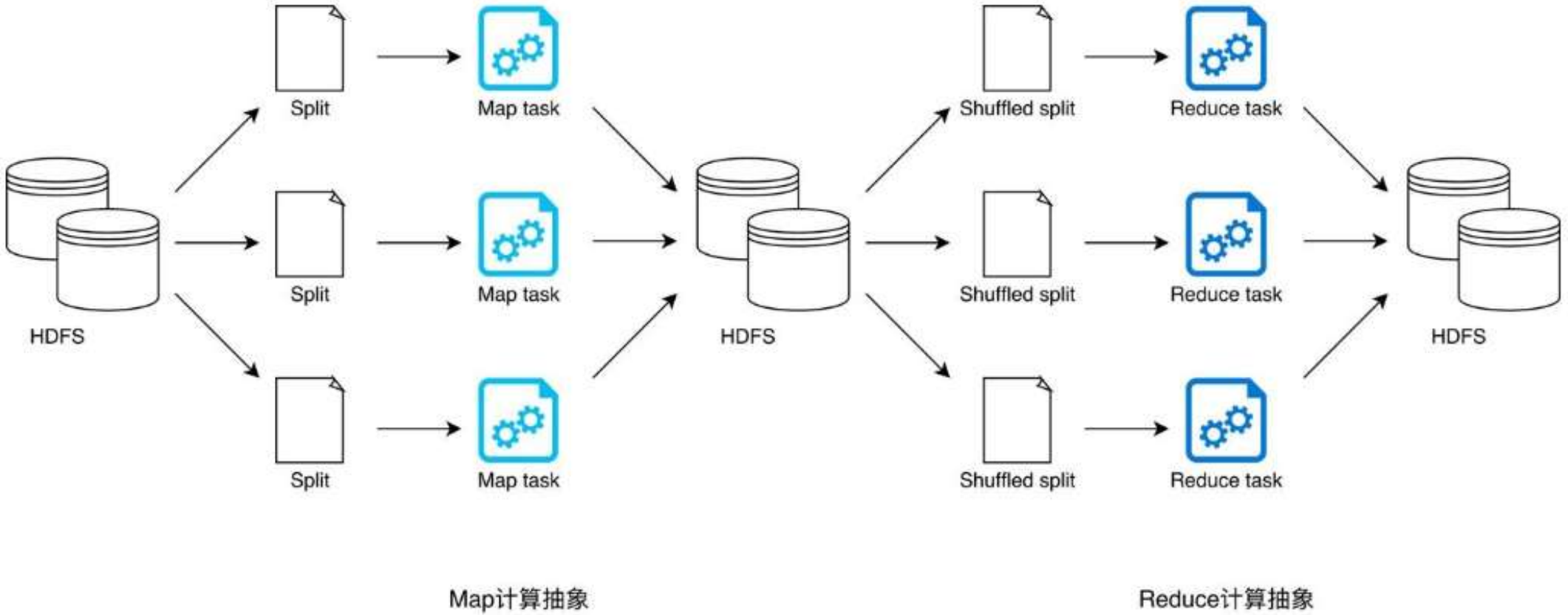
MapReduce每一个步骤发生在内存中，但产生的中间值（溢写文件）都会写入在磁盘里，下一步操作时又会将这个中间值merge到内存中，如此循环直到最终完成计算。

而对于**Spark**，每个步骤也是发生在内存之中，但**产生的中间值会直接进入下一个步骤**，直到所有的步骤完成之后才会将最终结果保存进磁盘。

所以在使用Spark做数据分析时，节省了大量的读写时间。当计算步骤很多时，Spark的优势就体现出来了

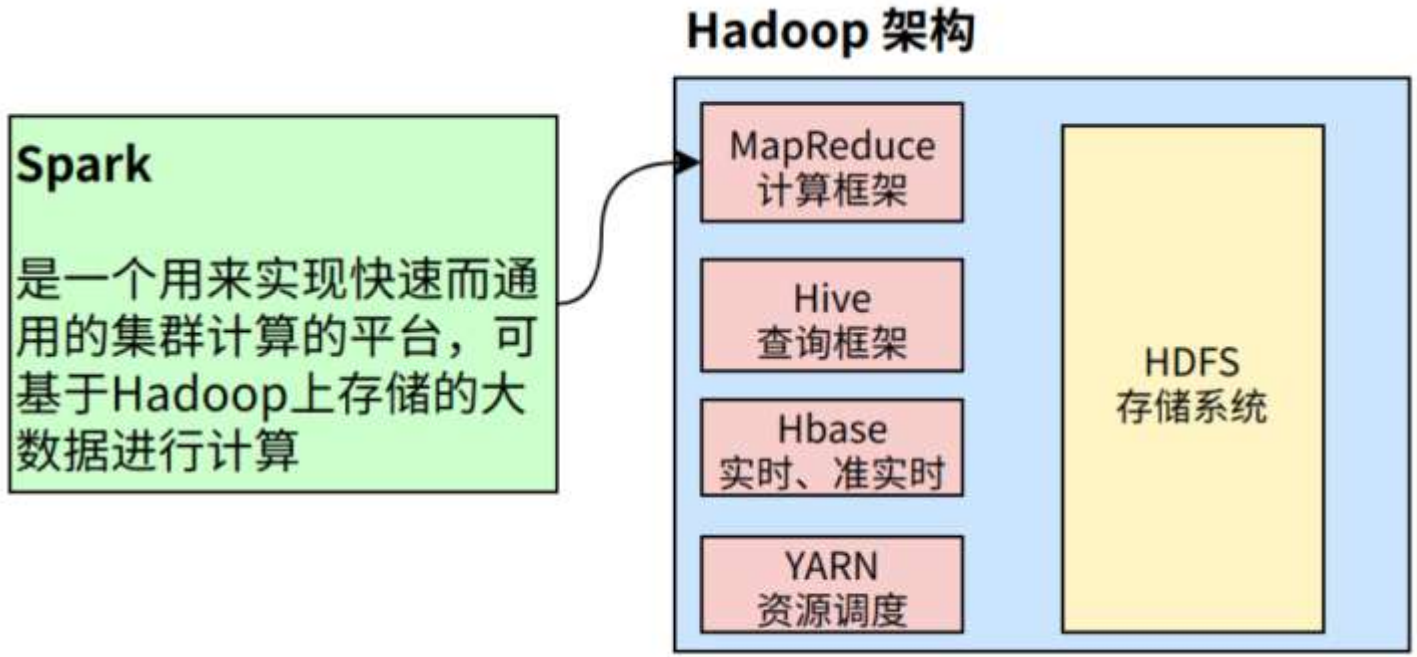
Recap: MapReduce的计算过程

在这个过程中进行了至少三次数据读写，Hadoop处理大数据的流程高度依赖磁盘读写，那么在数据处理上就出现了瓶颈，面对更复杂的计算逻辑的处理任务时，Hadoop存在很大局限性。



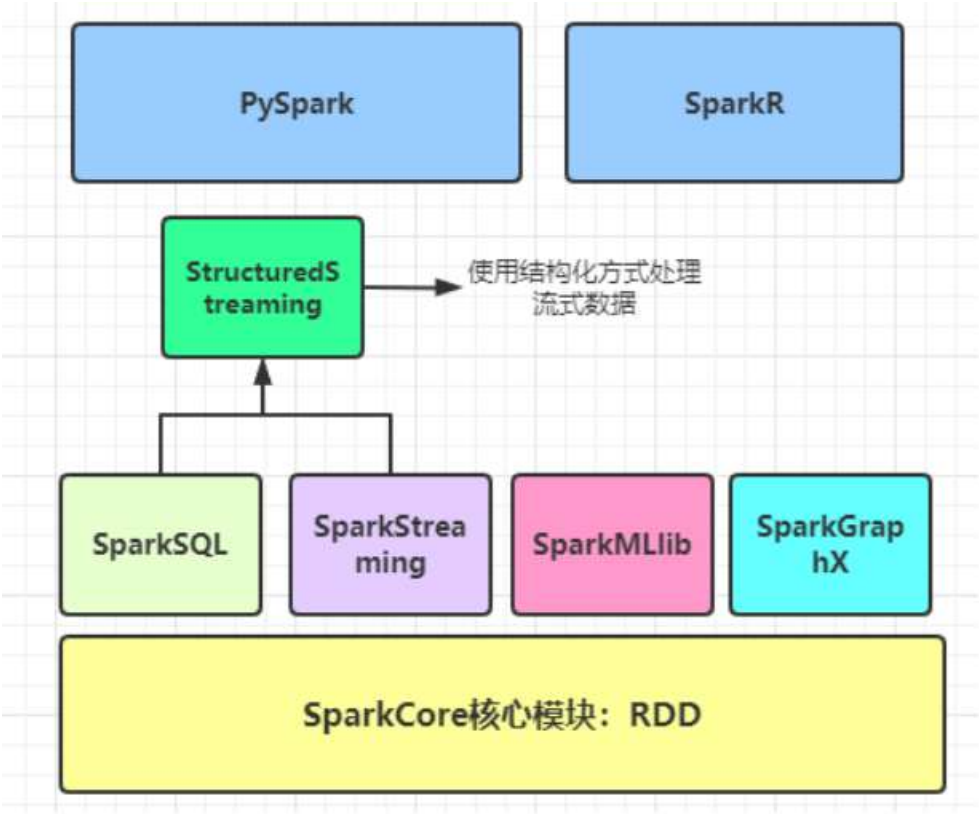
- **Spark在这样的背景下产生，它不像Hadoop一样采取磁盘读写，而是基于性能更高的内存存储来进行数据存储和读写**
  - 但是Spark缺乏对数据存储这一块的支持，**即没有分布式文件系统，必须依赖外部的数据源**，这个依赖可以是Hadoop系统的HDFS，也可以是其他的分布式文件系统，甚至可以是MySQL或本地文件系统。

Hadoop和Spark两者都是大数据框架，但是各自存在的目的不同。Hadoop实质上是一个**分布式数据基础设施**，它将巨大的数据集分派到一个集群中的多个节点**进行存储**，并具有**计算处理**的功能。Spark（**准确的说是SparkCore**）则不会进行分布式数据的**存储**，是**计算分布式数据的工具**



Spark框架可以针对任何业务类型分析进行处理，

- 支持离线批处理SparkCore
- 支持交互式分析的SparkSQL
- 支持流计算的SparkStreaming和StructuredStreamig
- 支持机器学习的MIlib
- 支持图计算的GraphX



上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 Spark简介

2 Spark编程模型

3 SparkSQL

RDD的概念

- **RDD是Spark的核心概念**，是弹性数据集（Resilient Distributed Datasets）的缩写。**RDD是分布式内存**的一个抽象概念，是只读的记录分区集合，能横跨集群所有节点进行并行计算。
- 我们在用MapReduce编程的时候，思考的是，如何将计算逻辑用 Map和Reduce两个阶段实现，map和reduce函数的输入和输出是什么
- 而Spark则**直接针对数据进行编程**，**将大规模数据集合抽象成一个RDD对象，然后在这个RDD上进行各种计算处理，得到一个新的RDD，并继续计算处理，直至得到最后的数据结果**
- 所以，**Spark可以理解成是面向对象的大数据计算。**我们在进行Spark编程的时候，主要思考的是**一个RDD对象需要经过什么样的操作，转换成另一个RDD对象**

RDD的概念

- 用大白话讲，RDD就是spark对数据的一系列操作，以方便分布式的内存计算
- Spark直接处理数据产生的中间值

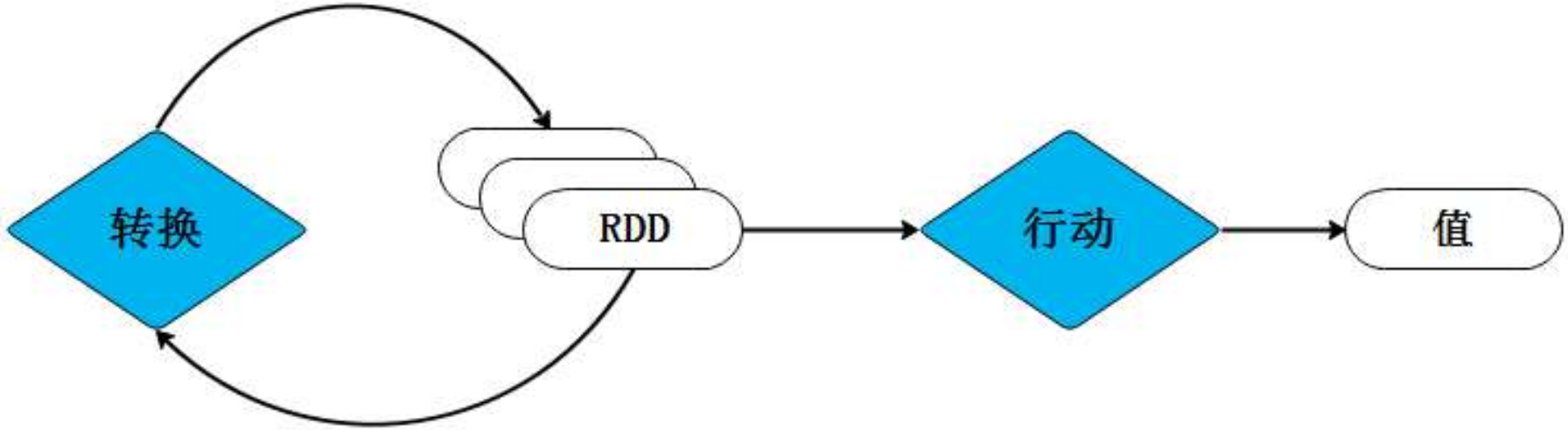
RDD提供了一组丰富的操作以支持常见的数据运算，分为“**行动**”（Action）和“**转换**”（Transformation）两种类型

行动算子	相关说明
count()	返回数据集中的元素个数
first()	返回数组的第一个元素
take(n)	以数组的形式返回数组集中的前n个元素
reduce(func)	通过函数func（输入两个参数并返回一个值）聚合数据集中的元素
collect()	以数组的形式返回数据集中的所有元素
foreach(func)	将数据集中的每个元素传递到函数func中运行

转换算子	相关说明
filter(func)	筛选出满足函数func的元素，并返回一个新的数据集
map(func)	将每个元素传递到函数func中，返回的结果是一个新的数据集
flatMap(func)	与map()相似，但是每个输入的元素都可以映射到0或者多个输出结果
groupByKey()	应用于（Key，Value）键值对的数据集时，返回一个新的（Key，Iterable <Value>）形式的数据集
reduceByKey(func)	应用于（Key，Value）键值对的数据集时，返回一个新的（Key，Value）形式的数据集。其中，每个Value值是将每个Key键传递到函数func中进行聚合后的结果

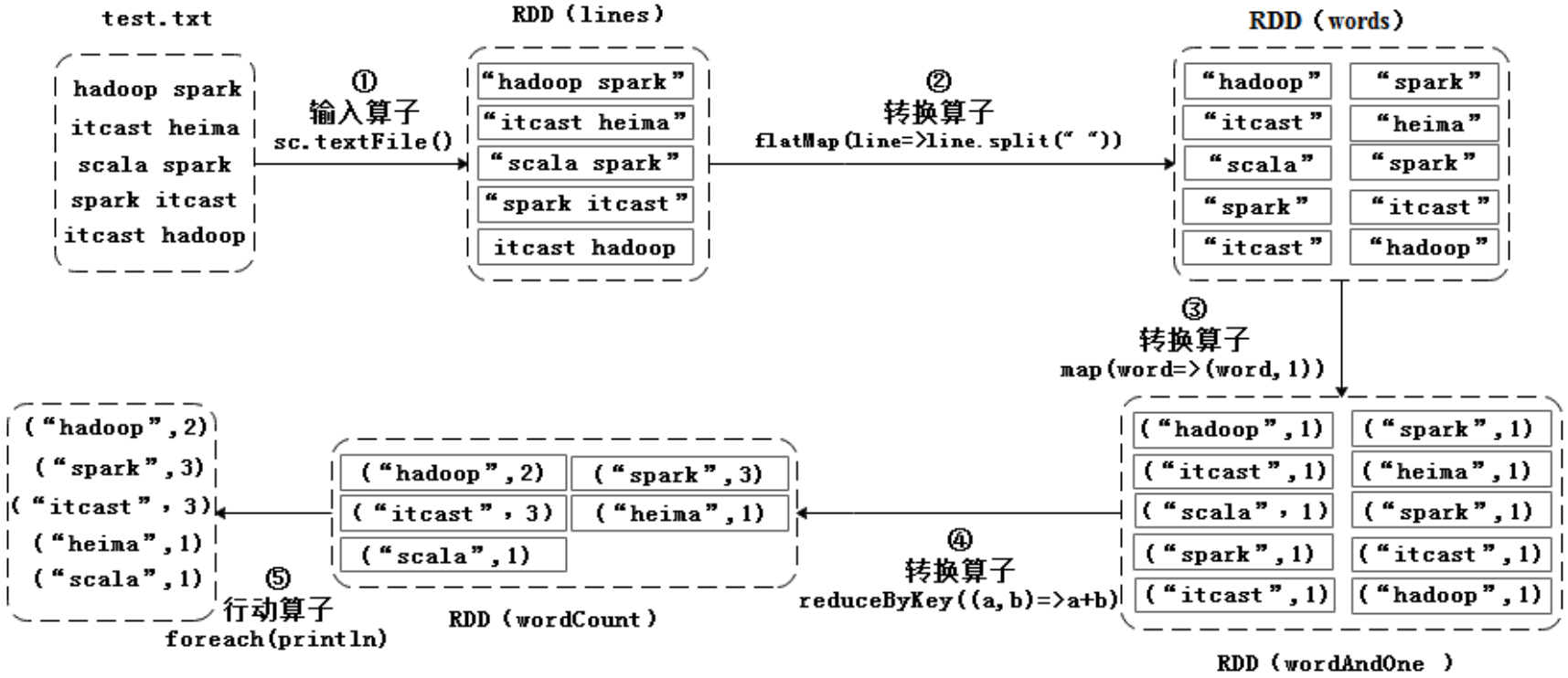
RDD的处理过程

- **RDD**经过一系列的**“转换”操作**，每一次转换都会产生不同的RDD，以供给下一次“转换”操作使用，直到最后一个RDD经过**“行动”操作**才会被**真正计算处理**，并输出到外部数据源中，若是**中间**的**数据结果**需要复用，则可以进行**缓存**处理，将数据缓存到**内存**中。



RDD的处理过程

- 例：WordCount词频统计
- 在Linux本地系统的/export/data目录下，有一个test.txt文件，文件里有多行文本，每行文本都是由2个单词构成，且单词之间都是用空格分隔。现在，我们需要通过RDD统计每个单词出现的次数（即词频），具体操作过程如下。

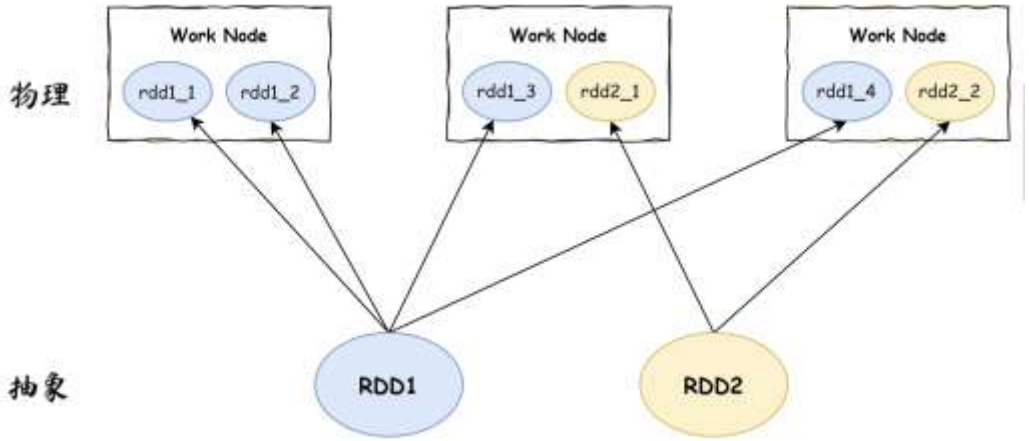


RDD的特性

- **分区：**一个RDD可以有很多分区，每一个分区内部是包含了该RDD的部分数据
- **可并行计算：**Spark中RDD的计算是以分区为单位的，每个RDD都会实现compute计算函数以达到这个目的
- **依赖关系：**一个RDD会依赖于其他多个RDD，Spark可以根据这些依赖关系重建RDD，是一种容错机制
- **RDD分区函数（可选项）：**只有对于key-value的，RDD并且产生shuffle，才有分区器
- **优先位置列表（可选项）：**spark任务在调度的时候会优先考虑存有数据的节点开启计算任务，减少数据的网络传输，提升计算效率

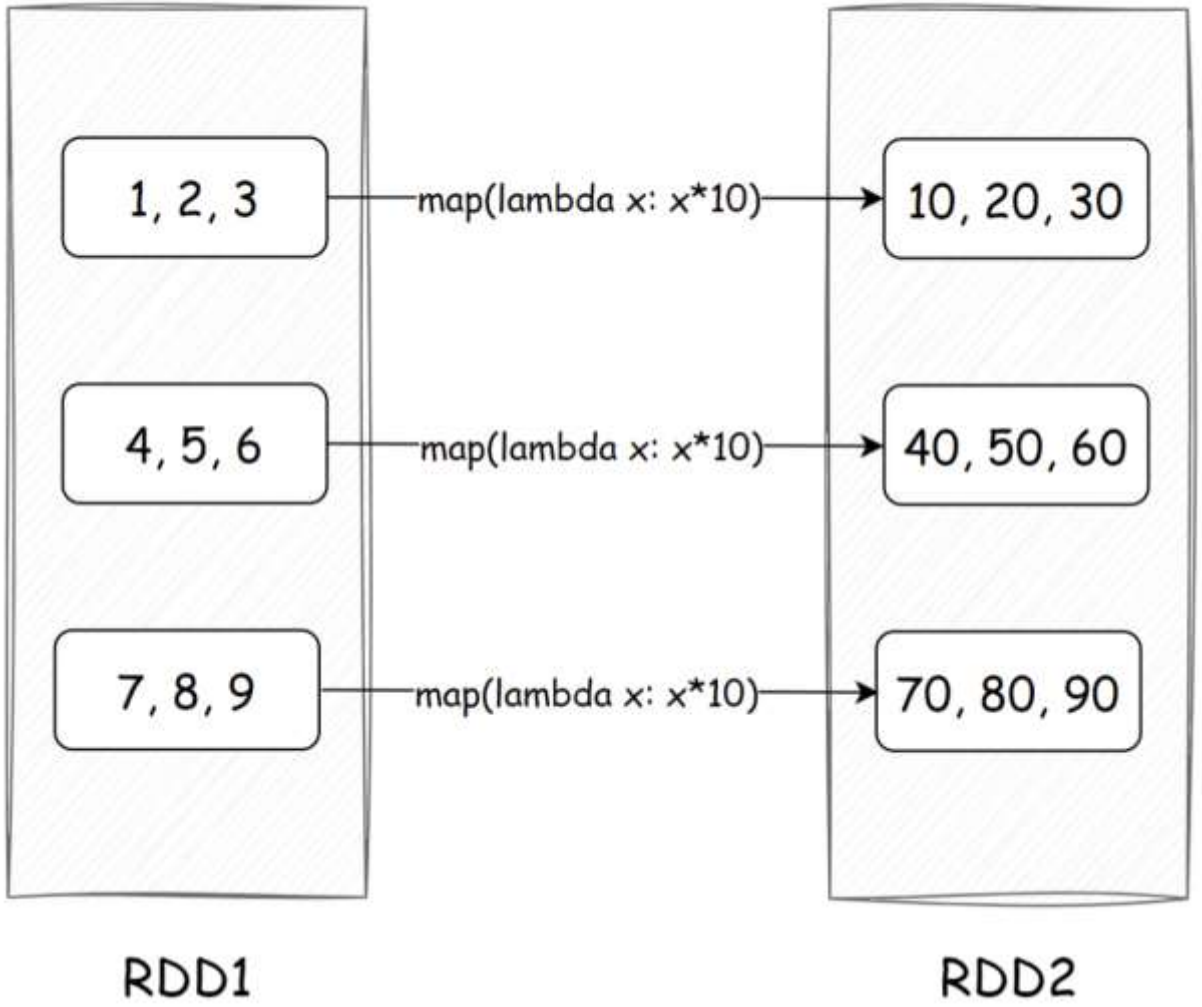
RDD的分区

- 一个RDD有很多分区，每一个分区内部是包含了该RDD的部分数据，spark中任务是以task线程的方式运行， 一个分区就对应一个task线程

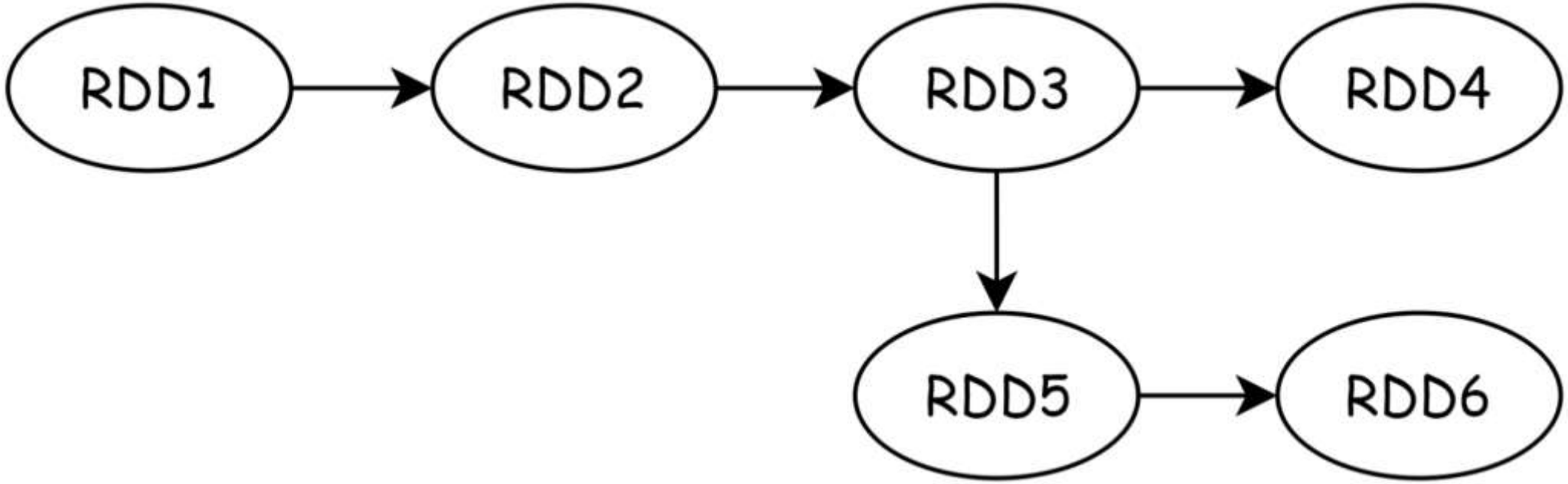


- RDD的分区原则是分区的个数尽量等于集群中的CPU核心（Core）数目。对于不同的Spark部署模式而言，都可以通过参数值来配置默认的分区数目

RDD的可并行计算

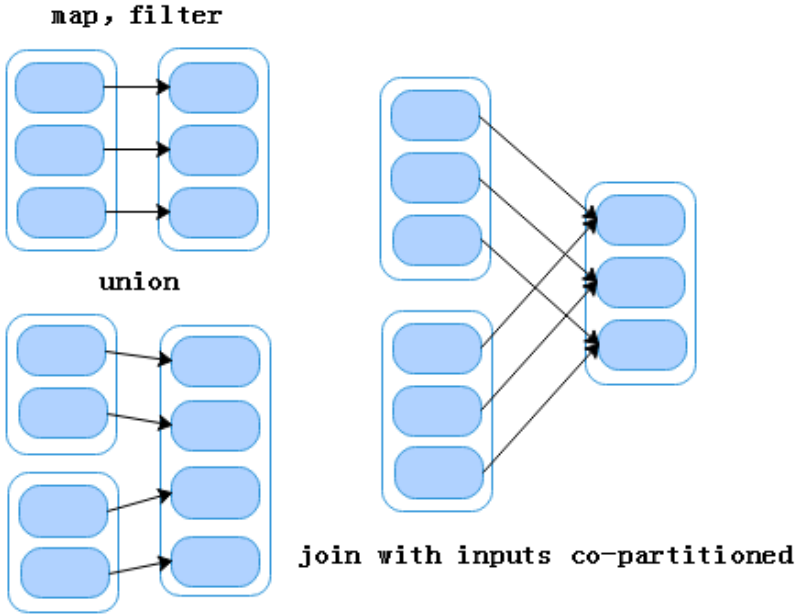

1, 2, 3
map(lambda x: x*10)
10, 20, 30
4, 5, 6
map(lambda x: x*10)
40, 50, 60
7, 8, 9
map(lambda x: x*10)
70, 80, 90
RDD1
RDD2

RDD的依赖关系

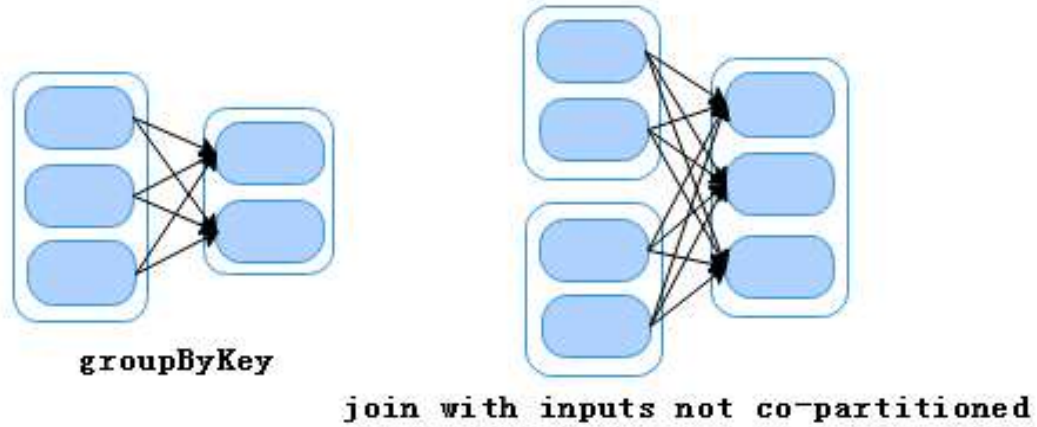


RDD的依赖关系

- **窄依赖**表现为一个父RDD的分区对应于一个子RDD的分区，或者多个父RDD的分区对应于一个子RDD的分区
- **宽依赖**则表现为存在一个父RDD的一个分区对应一个子RDD的多个分区



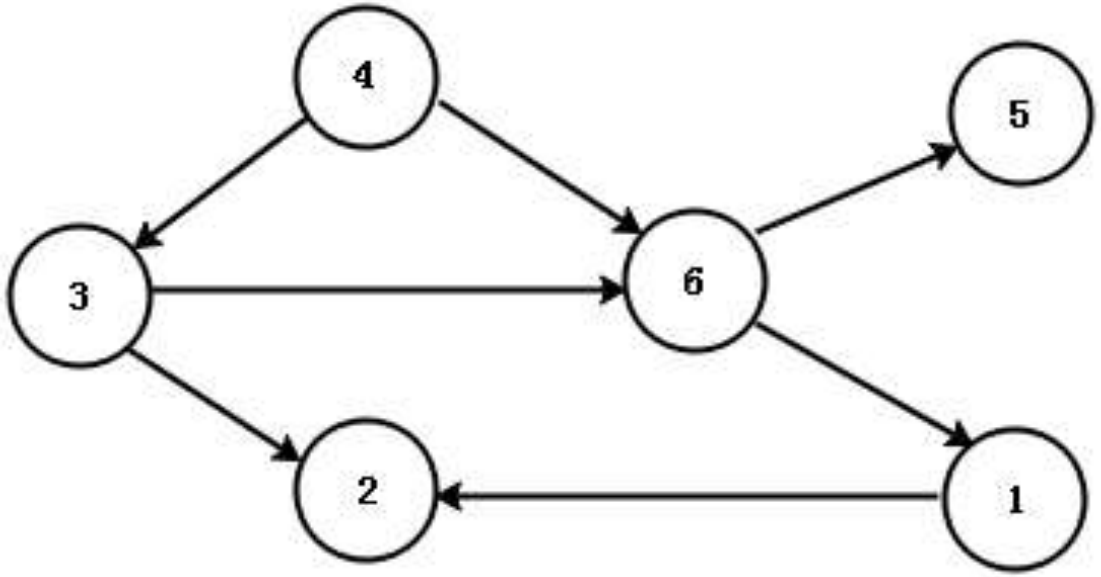
窄依赖



宽依赖

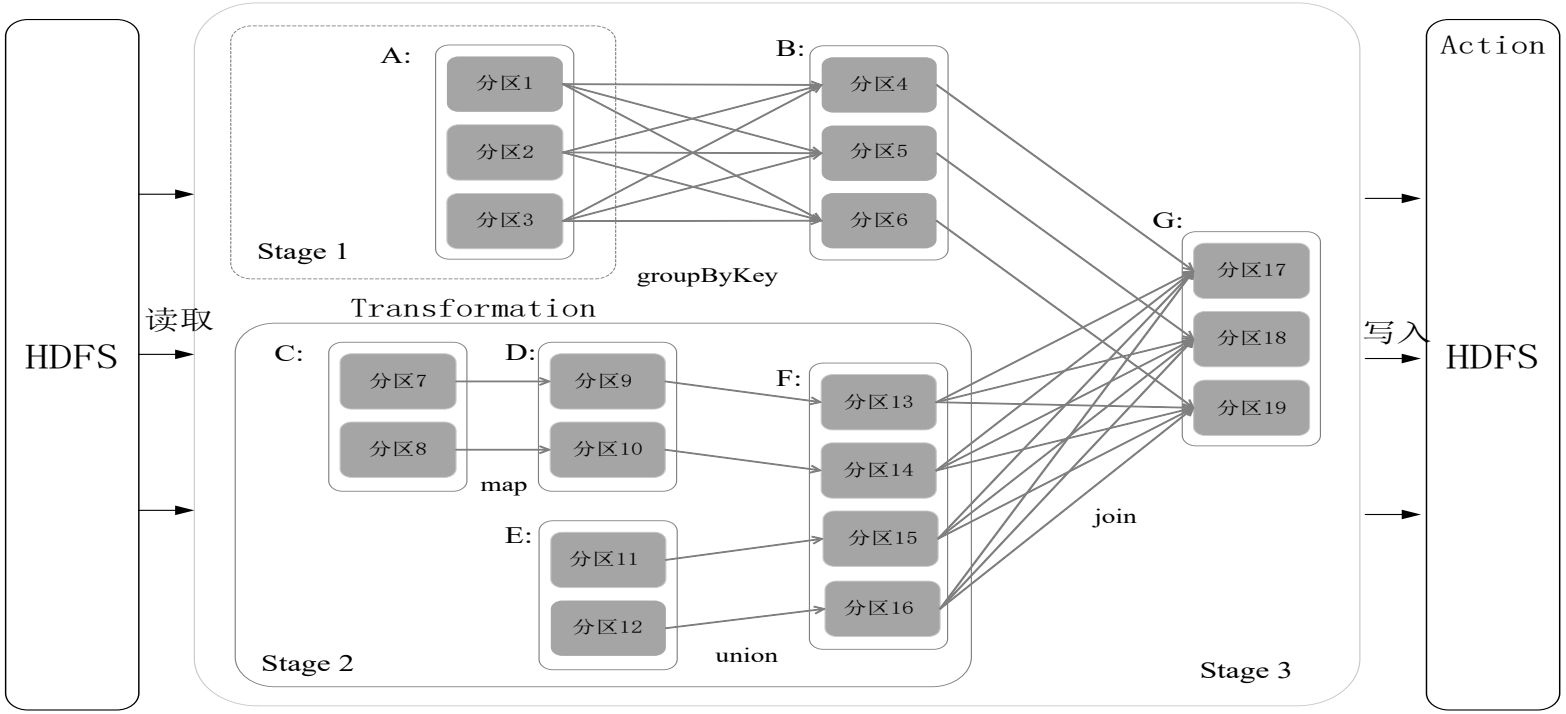
DAG的概念

- **DAG**（Directed Acyclic Graph）叫做有向无环图，Spark中的RDD通过一系列的转换算子操作和行动算子操作形成了一个DAG。DAG是一种非常重要的**图论数据结构**。如果一个有向图无法从任意顶点出发经过若干条边回到该点，则这个图就是有向无环图。“4→6→1→2”是一条路径，“4→6→5”也是一条路径，并且图中不存在从顶点经过若干条边后能回到该点。



DAG的概念

- 根据**RDD之间依赖关系**的不同可将DAG划分成不同的**Stage**(调度阶段)。
- 对窄依赖来说，RDD分区的转换处理是在一个线程里完成，所以窄依赖会被Spark划分到同一个Stage中
- 而对宽依赖来说，由于有Shuffle存在，所以只能在父RDD处理完成后，下一个Stage才能开始接下来的计算，因此**宽依赖**是**划分Stage**的**依据**，当RDD进行转换操作，遇到宽依赖类型的转换操作时，就划为一个Stage。

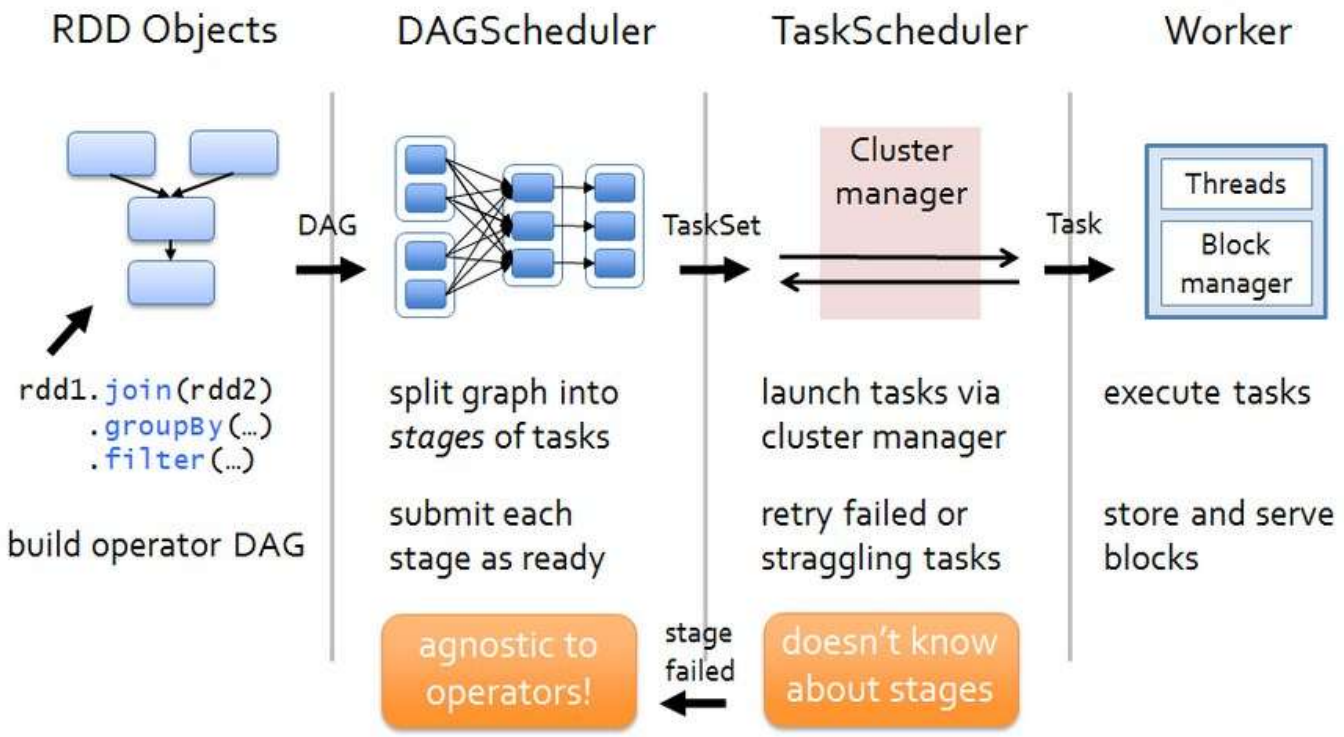


RDD在Spark中的运行过程

（1）创建RDD对象；

（2）SparkContext负责计算RDD之间的依赖关系，构建DAG；

（3）DAGScheduler负责把DAG图分解成多个Stage，每个Stage中包含了多个Task，每个Task会被TaskScheduler分发给各个WorkerNode上的Executor去执行



上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 Spark简介

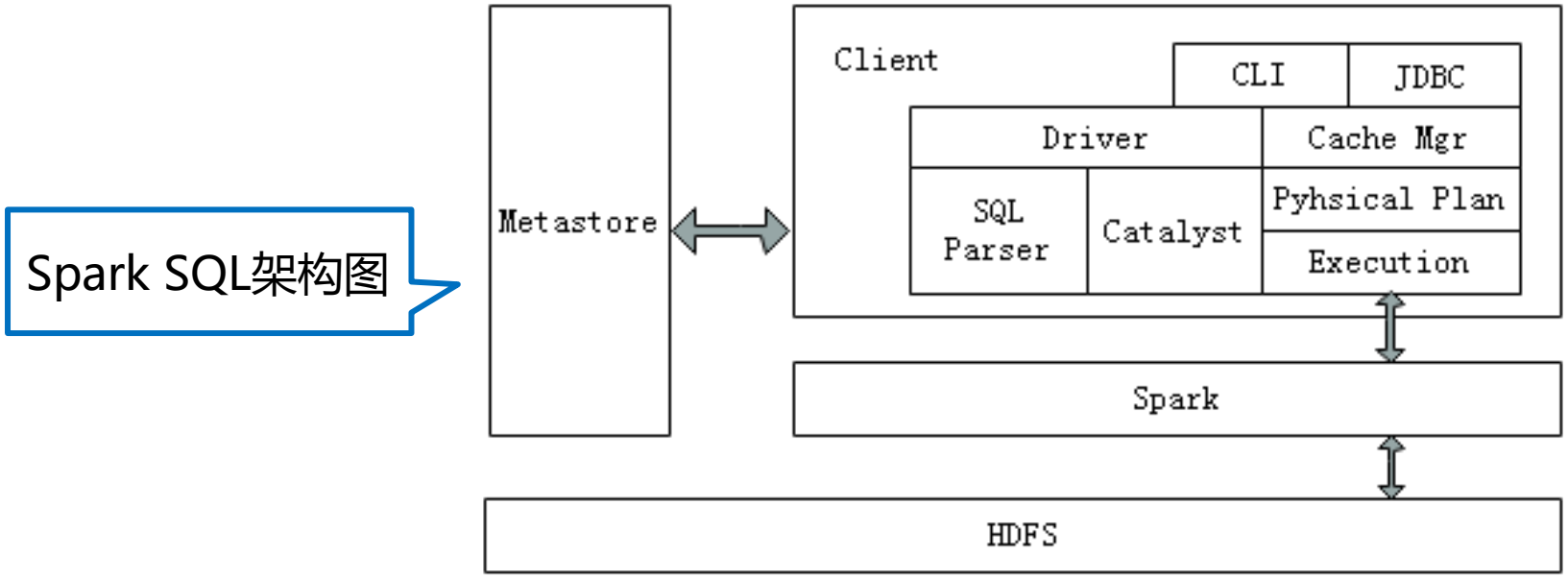
2 Spark编程模型

3 SparkSQL

- Spark SQL是Spark用来处理结构化数据的一个模块，它提供了一个编程抽象结构叫做DataFrame的数据模型（即带有Schema信息的RDD），Spark SQL作为分布式SQL查询引擎，让用户可以通过SQL、DataFrames API和Datasets API三种方式实现对结构化数据的处理。

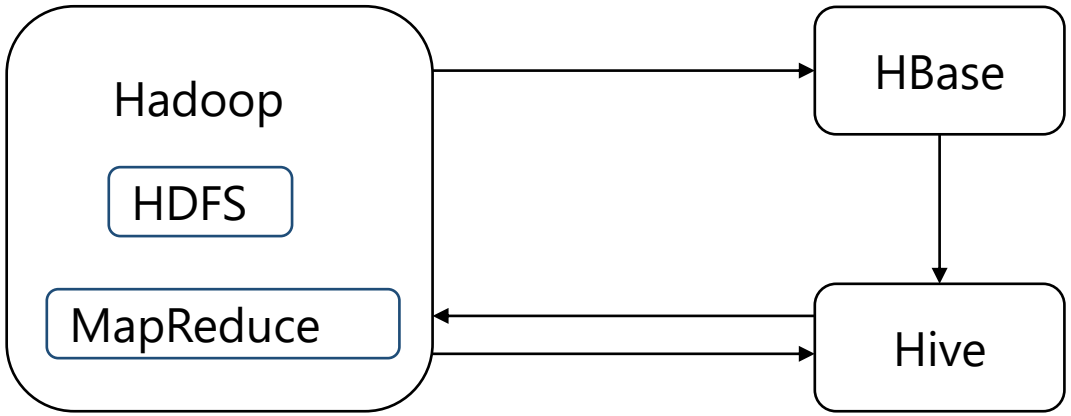
SparkSQL架构

- Spark SQL架构与Hive架构相比，把底层的MapReduce执行引擎更改为Spark，还修改了Catalyst优化器，Spark SQL快速的计算效率得益于Catalyst优化器。从HiveQL被解析成语法抽象树起，执行计划生成和优化的工作全部交给Spark SQL的Catalyst优化器进行负责和管理。



Recap: HDFS，HBase和Hive

Hadoop提供基础架构，其中HDFS解决如何存的问题，MapReduce解决如何算的问题

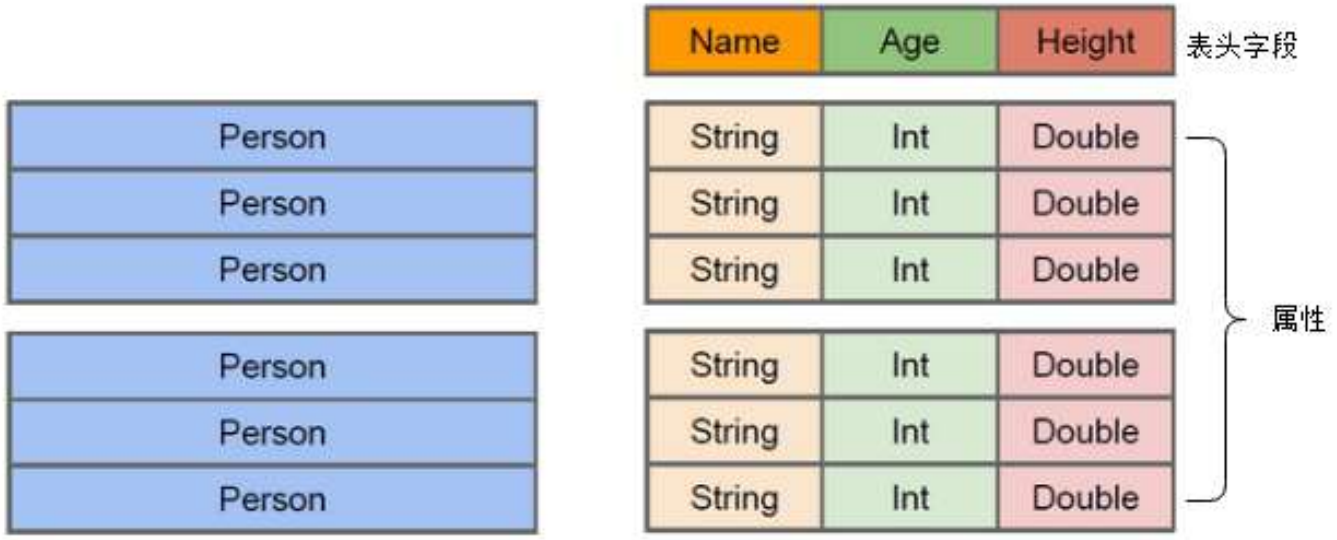


HBase解决如何查的问题，但是**不能直接访问HDFS的接口**，只能通过**构建自己的数据库，其存储仍然依赖HDFS**，可以进行简单查询，但是要实现类似SQL的复杂查询需要配合其他组件

Hive可以直接**通过构建外部表和HiveQL对HDFS查询**，**也可以在HBase的基础上实现标准SQL查**询，适用于复杂的数据仓库分析和查询

DataFrame

- Spark SQL使用的数据抽象并非是RDD，而是DataFrame。
- DataFrame的结构类似传统数据库的二维表格，可以从很多数据源中创建，如结构化文件、外部数据库、Hive表等数据源。
- DataFrame可以看作是分布式的Row对象的集合，在二维表数据集的每一列都带有名称和类型，这就是Schema元信息



SQL风格操作DataFrame

- 例：**查询年龄最大的前两名人的信息**

```
scala > spark.sql("select * from t_person order by age desc limit 2").show()
+---+------+---+
| id|  name|age|
+---+------+---+
|  6| jerry| 40|
|  5|tianqi| 35|
+---+------+---+
```

SQL风格操作DataFrame

- 例：**查询年龄大于25的人的信息**

```
scala > spark.sql("select * from t_person where age > 25").show()
+---+-------+---+
| id   |  name |age|
+---+-------+---+
|  2   |   lisi     | 29  |
|  4   | zhaoliu| 30 |
|  5   | tianqi  | 35  |
|  6   |  jerry   | 40  |
+---+-------+---+
```

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

Thank You

